From OOP State to Dynamic Data Structures

CMPT 211: Introduction to Software Development

Bernal Manzanilla Saavedra

Concordia University of Edmonton

Winter 2026

The Limitation of Flat Lists

Last week, we imagined storing student grades in a flat list: `self.grades = [85, 92, 78]`.

The Analytical Flaw: A flat list loses all contextual mapping. We know the student achieved an 85%, but *which course* does that belong to? Was it CMPT 211 or an elective?

The Solution: Key-Value Mapping We must upgrade the internal state of our `ConcordiaStudent` class from a 1-Dimensional List to a **Dictionary**. A dictionary explicitly maps a unique **Key** (the Course Name) to a specific **Value** (the Grade).

Step 1: The add_grade Method Explained

Let's rewrite our Base Class constructor and our Setter method to utilize a dictionary securely.

```
class ConcordiaStudent:
    def __init__(self, name, student_id):
        self.name = name
        self.student_id = student_id
        # UPGRADE: Initialize an empty dictionary instead of a list
        self.courses = {}

    def add_grade(self, course_name, grade_percentage):
        # Step 1a: ENCAPSULATION SHIELD. Validate the data first.
        if 0 <= grade_percentage <= 100:
            # Step 1b: DICTIONARY INSERTION.
            # We access the dictionary, define the new Key inside the
            # brackets [course_name], and assign the Value.
            self.courses[course_name] = grade_percentage
        else:
            raise ValueError("Grade must be between 0 and 100.")
```

Step 2: The Dynamic GPA Calculator

How do we calculate an average when our data is trapped inside a dictionary like {"Math": 80, "CMPT": 90}? We must extract *only the values*.

```
    def calculate_gpa(self):
        # Step 2a: Prevent Division by Zero if the student has no courses
        if len(self.courses) == 0:
            return 0.0

        # Step 2b: Extract ONLY the numerical grades.
        # .values() strips away the course names, leaving just the numbers.
        all_grades = self.courses.values()

        # Step 2c: Math operations. sum() adds them up, len() counts them.
        average_percentage = sum(all_grades) / len(all_grades)

        # Step 2d: Convert the 100-point scale to a 4.0 scale.
        gpa = (average_percentage / 100) * 4.0
        return round(gpa, 2)
```

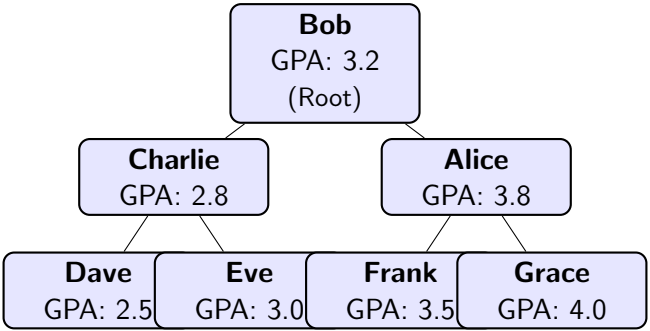
Bridging to Data Structures: The Ranking Problem

Now we have Alice and Bob. They take the exact same courses, and our OOP methods calculate their GPAs dynamically.

The Administrative Problem: The Dean wants a printed list of all 5,000 Concordia students, **ranked in order from lowest GPA to highest GPA**.
If we put 5,000 students in a standard Python `list` and try to sort them, it is computationally expensive ($O(n \log n)$).
The Solution: A Binary Search Tree (BST) As we create each `ConcordiaStudent` object, we instantly place them into a Tree structure based on their GPA.

Visualizing the GPA Ranking Tree (BST)

The Rule: Left Child is a lower GPA. Right Child is a higher GPA.



To print the ranked list, we simply perform an **In-Order Traversal** *(Left → Root → Right). The output automatically prints: Dave (2.5), Charlie (2.8), Eve (3.0), Bob (3.2), Frank (3.5), Alice (3.8), Grace (4.0).*

Another Administrative Example: The Waitlist Heap

Binary Trees are used for more than just search rankings!

The Scenario: A popular CMPT course is full. 50 students are on the waitlist. When a seat opens up, who gets it? A standard Queue (First-Come, First-Served) is unfair to 4th-year students who desperately need the course to graduate.

The Solution: A Priority Queue (Max-Heap)

- A Heap is a Tree where the "highest priority" node always floats to the very top (the Root).
- We can assign a priority score to our `ConcordiaStudent` objects based on their `self.year`.
- When a seat opens, the system simply "Pops" the Root of the tree, automatically granting the seat to the 4th-year student, regardless of when they joined the waitlist!

Dynamic Data Structures (CLRS Chapters 6-10)

In the *Introduction to Algorithms* text (CLRS), dynamic structures like Trees and Linked Lists are explained using **memory arrays and pointers**.

Python has no explicit "Pointers". How do we replicate this textbook material?

- Instead of hexadecimal memory addresses, we will use **List Indices** as our pointers.
- We will represent individual Tree Nodes as small Python lists.
- By storing these small lists inside a larger "Memory Pool" list, we can link them together dynamically, exactly how lower-level languages operate in RAM!

Simulating Node Pointers in Python

We will represent a **Binary Tree Node** as a 4-element list:

Format: `[ Student_Object, Parent_Index, Left_Index, Right_Index ]`
(We use `None` to represent a NULL pointer, meaning the branch ends).

```
# Our "Memory Pool" representing the computer's RAM
ram_memory = []

# Node 0 (The Root: Bob)
# Parent is None. Left child is at Index 1. Right child is at Index 2.
root_node = [bob_obj, None, 1, 2]
ram_memory.append(root_node)

# Node 1 (Left Child: Charlie)
left_node = [charlie_obj, 0, None, None] # Points back up to Parent (0)
ram_memory.append(left_node)

# Node 2 (Right Child: Alice)
right_node = [alice_obj, 0, None, None] # Points back up to Parent (0)
ram_memory.append(right_node)
```

Traversing the Pointer Array

How do we traverse this structure? We use the indices as "hops" to jump around the memory array!

```
# Let's find out who the Root's right child is!

# 1. We start at the root (Index 0 in memory)
current_idx = 0
current_node = ram_memory[current_idx] # [bob_obj, None, 1, 2]

# 2. We look at the "Right_Index" field (Index 3 of the node list)
right_child_idx = current_node[3] # This is 2

# 3. We "jump" to that index in our RAM array!
next_node = ram_memory[right_child_idx] # [alice_obj, 0, None, None]

# 4. We extract the Student Object (Index 0) and check her GPA!
student_record = next_node[0]
print(f"Right child is {student_record.name} with GPA {student_record.calculate_gpa
    ()}")
# Output: Right child is Alice with GPA 3.8
```

Summary

By grouping data into small, structured lists `[Val, Top, Left, Right]`, we can perfectly translate the low-level pointer mechanics of the CLRS textbook into readable, executable Python code!